Web & apps

Javascript

Konstantinos Tserpes
tserpes@mail.ntua.gr

Introduction

- Scripting language
- Code is parsed in a “runtime” (as opposed to “compiler”)
  - Google V8 engine
  - Mozilla SpiderMonkey
- Specification: ECMAScript
  - Course emphasis on ECMAScript 6 (ES6)

JavaScript (JS) Execution model/Runtime

- JavaScript engine
  - Google V8
  - Mozilla SpiderMonkey
  - Apple JavaScriptCore
  - MS Chakra (MS Edge now uses V8)
- host environment
  - Browser HTML DOM
  - Node.js

JS Engine

- Software
- Implements the ECMAScript (JavaScript) language
- Agent: autonomous executor of JS
  - Heap (of objects)
  - Queue (of jobs) or event loop
  - Stack (of execution contexts)

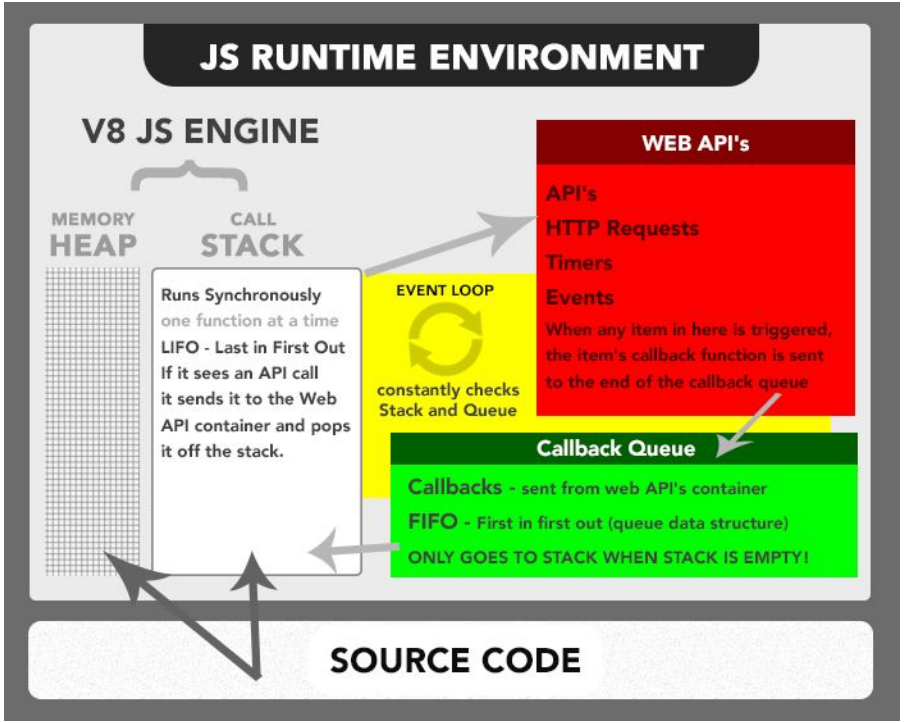
Programming language and OS recap

```
let x = 8;
function f(a,b){
    const c = 5;
    const d = a+b+c+x;
    return d;
}
function g(a){
    return f(a,a+3);
}
g(1);
f(3,4);
```

- realms
- execution context
- arguments
- variables
- bindings
- frames
- stack
- …

JS Runtime: V8

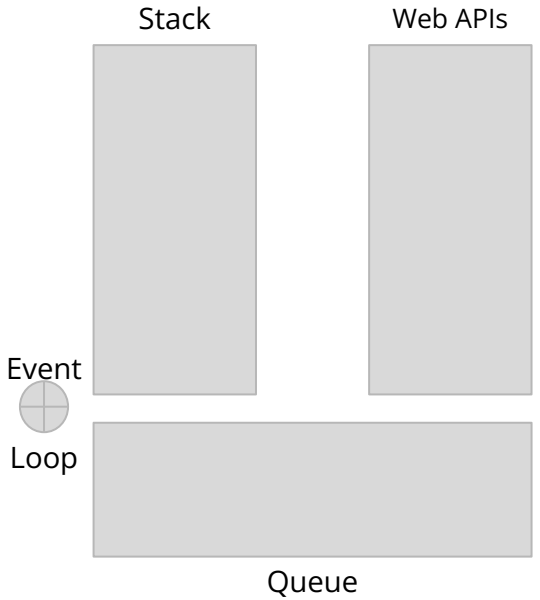
- Event-driven programming
  - Single threaded
    - predictable execution
    - no race conditions on the DOM
    - simpler programming model
  - Asynchronous
    - Reactionary code execution
  - Non-blocking I/Os
    - I/O operations that do not block the execution of other code



Source: https://medium.com/@olinations/the-javascript-runtime-environment-d58fa2e60dd0

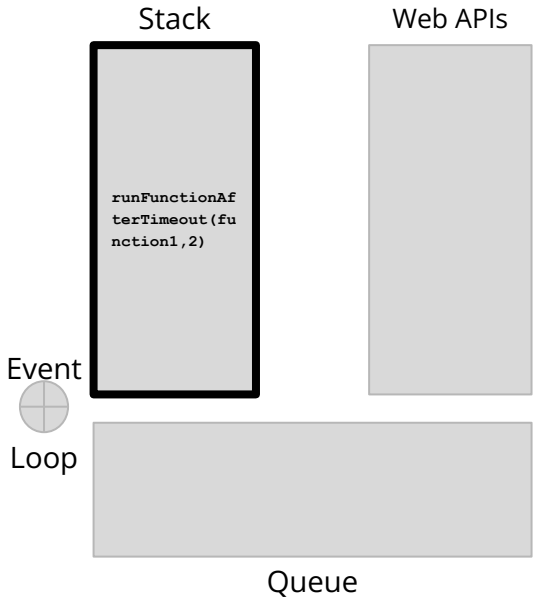
Example

```
function1{
      //code1
}
runFunctionAfterTimeout(function1,2); //timeout = 2’’; assume a really slow machine
function2(){
      //code2
}
function3(){
      //code3
}

function2();
function3();
```



Example

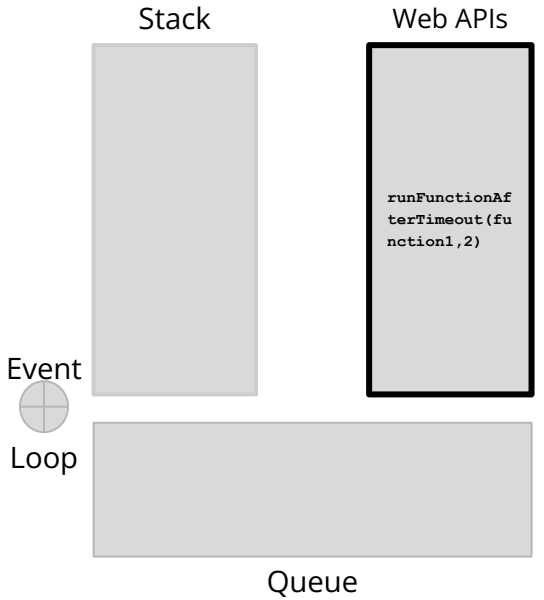
```
function1{
      //code1
}
runFunctionAfterTimeout(function1,2); //timeout = 2’’; assume a really slow machine
function2(){
      //code2
}
function3(){
      //code3
}

function2();
function3();
```



Example

t=0

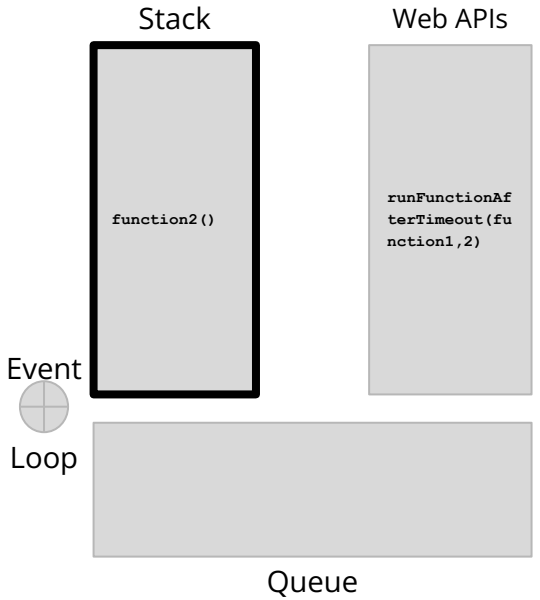
```
function1{
      //code1
}
runFunctionAfterTimeout(function1,2); //timeout = 2''; assume a really slow machine
function2(){
      //code2
}
function3(){
      //code3
}

function2();
function3();
```



Example

t=1

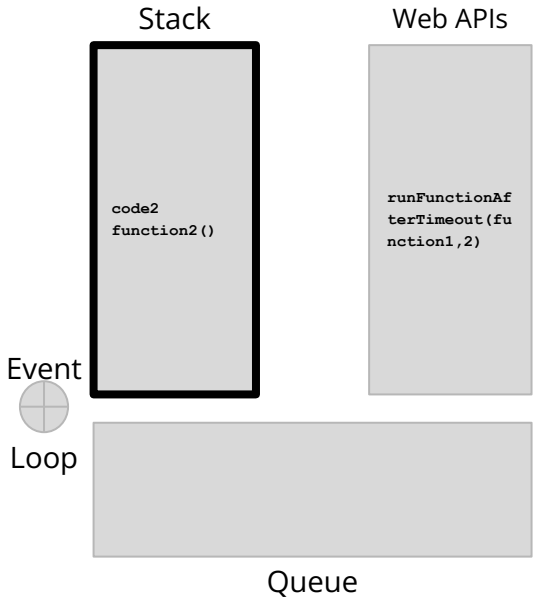
```
function1{
      //code1
}
runFunctionAfterTimeout(function1,2); //timeout = 2’’; assume a really slow machine
function2(){
      //code2
}
function3(){
      //code3
}

function2();
function3();
```



Example

t=1

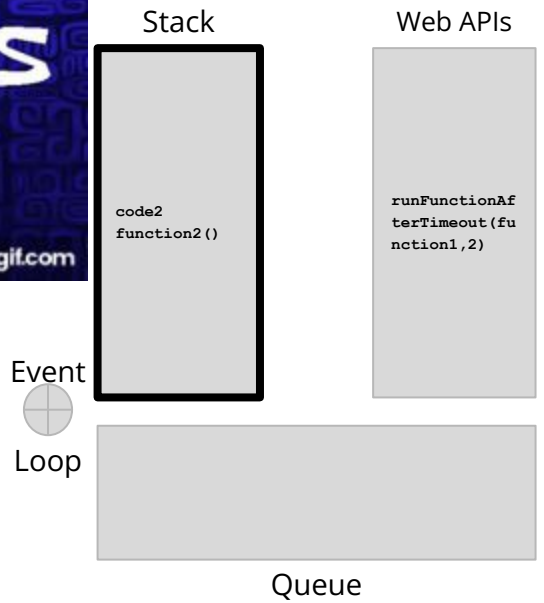
```
function1{
      //code1
}
runFunctionAfterTimeout(function1,2); //timeout = 2’’; assume a really slow machine
function2(){
      //code2
}
function3(){
      //code3
}

function2();
function3();
```



Example

t=1

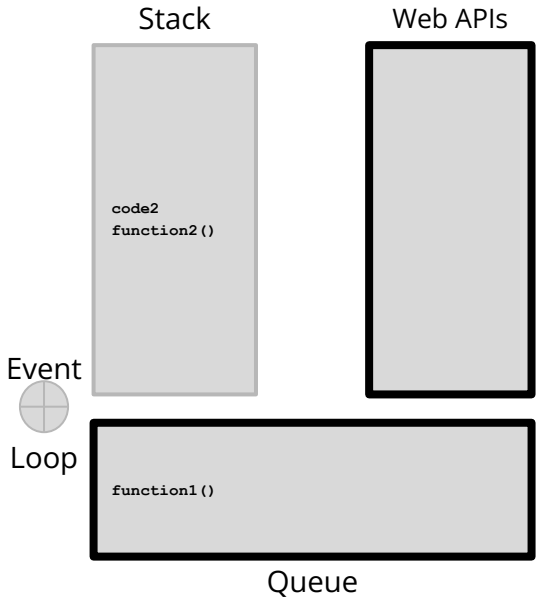
```
function1{
      //code1
}
runFunctionAfterTimeout(functio                                    ly slow machine
function2(){
      //code2
}
function3(){
      //code3
}

function2();
function3();
```



Example

t=2

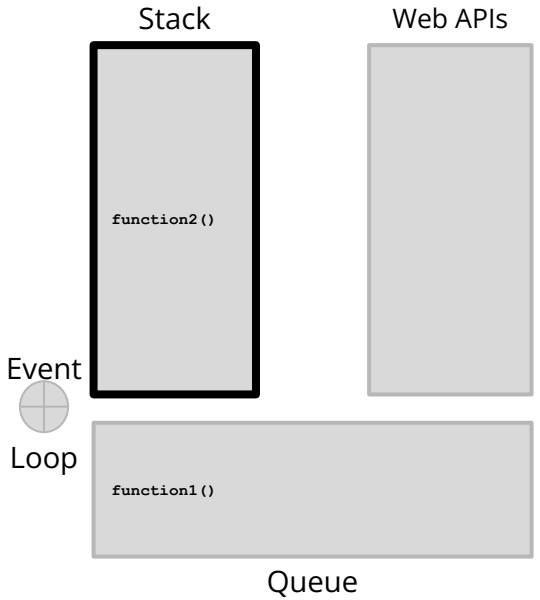
```
function1{
      //code1
}
runFunctionAfterTimeout(function1,2); //timeout = 2''; assume a really slow machine
function2(){
      //code2
}
function3(){
      //code3
}

function2();
function3();
```



Example

t=-

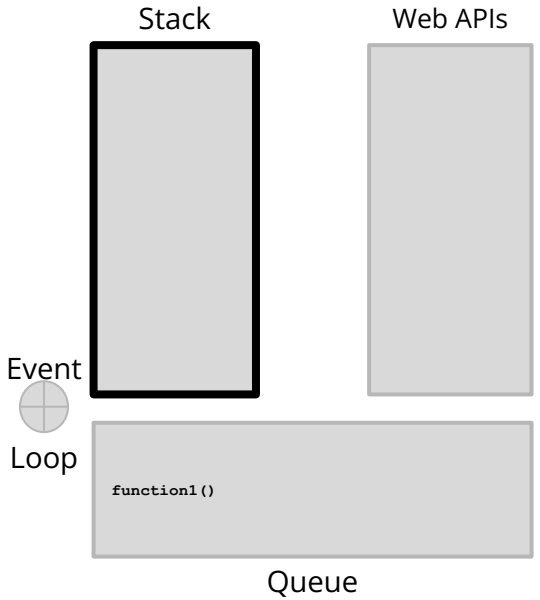
```
function1{
      //code1
}
runFunctionAfterTimeout(function1,2); //timeout = 2’’; assume a really slow machine
function2(){
      //code2
}
function3(){
      //code3
}

function2();
function3();
```



Example

t=-

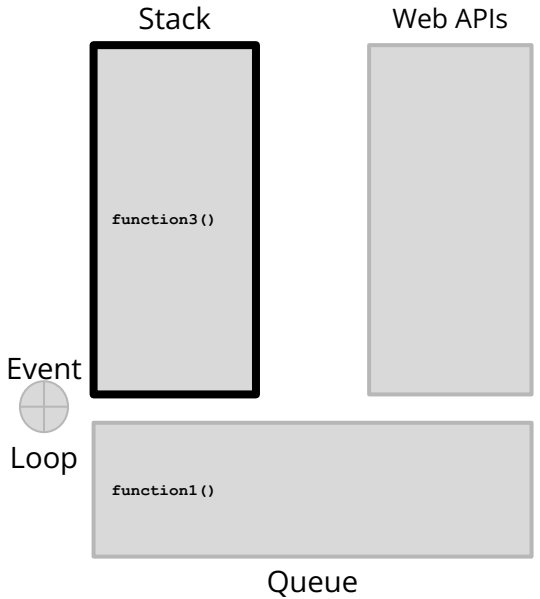
```
function1{
      //code1
}
runFunctionAfterTimeout(function1,2); //timeout = 2’’; assume a really slow machine
function2(){
      //code2
}
function3(){
      //code3
}

function2();
function3();
```



Example

t=-

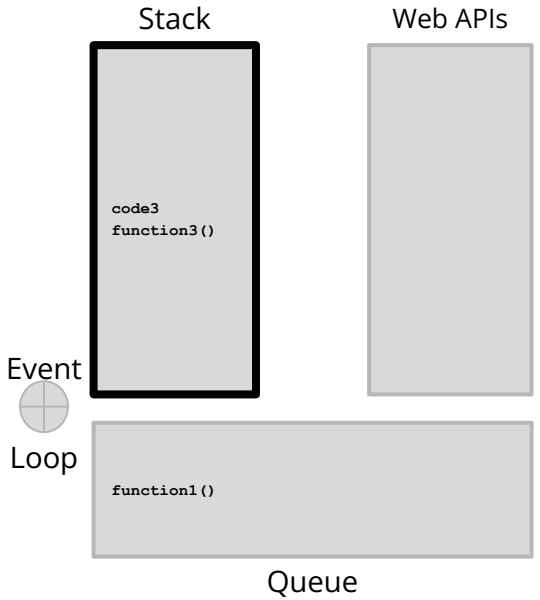
```
function1{
      //code1
}
runFunctionAfterTimeout(function1,2); //timeout = 2’’; assume a really slow machine
function2(){
      //code2
}
function3(){
      //code3
}

function2();
function3();
```



Example

t=-

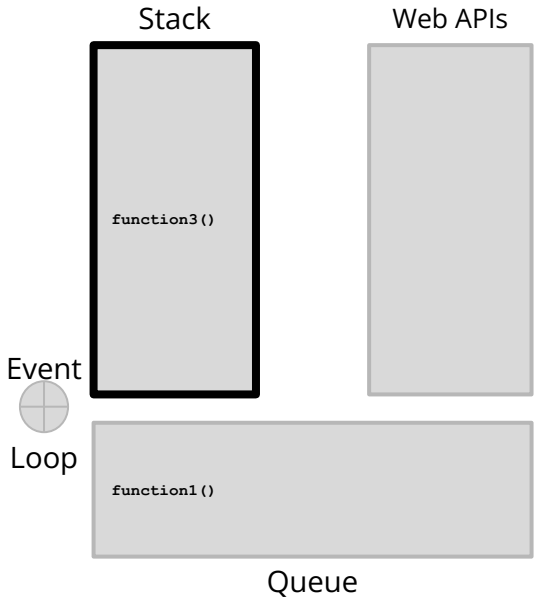
```
function1{
      //code1
}
runFunctionAfterTimeout(function1,2); //timeout = 2’’; assume a really slow machine
function2(){
      //code2
}
function3(){
      //code3
}

function2();
function3();
```



Example

t=-

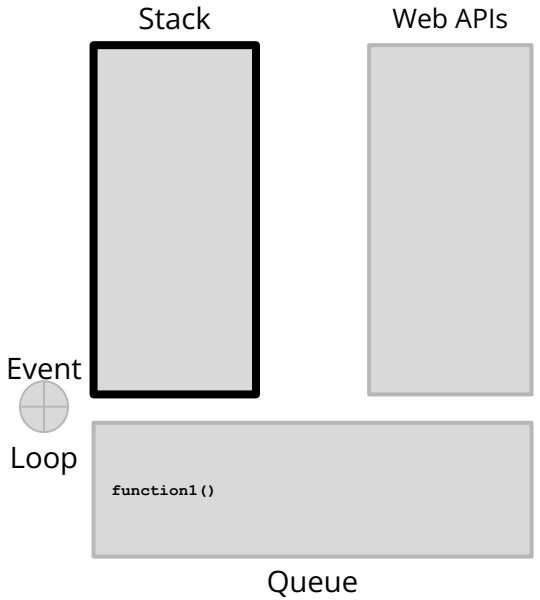
```
function1{
      //code1
}
runFunctionAfterTimeout(function1,2); //timeout = 2’’; assume a really slow machine
function2(){
      //code2
}
function3(){
      //code2
}

function2();
function3();
```



Example

t=-

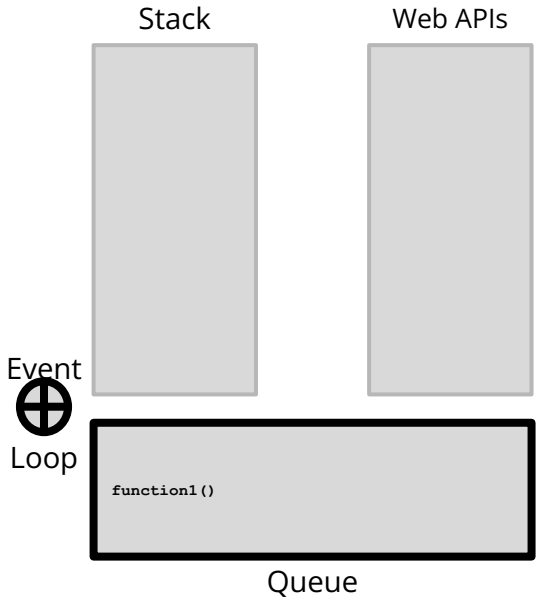
```
function1{
      //code1
}
runFunctionAfterTimeout(function1,2); //timeout = 2’’; assume a really slow machine
function2(){
      //code2
}
function3(){
      //code2
}

function2();
function3();
```



Example

t=-

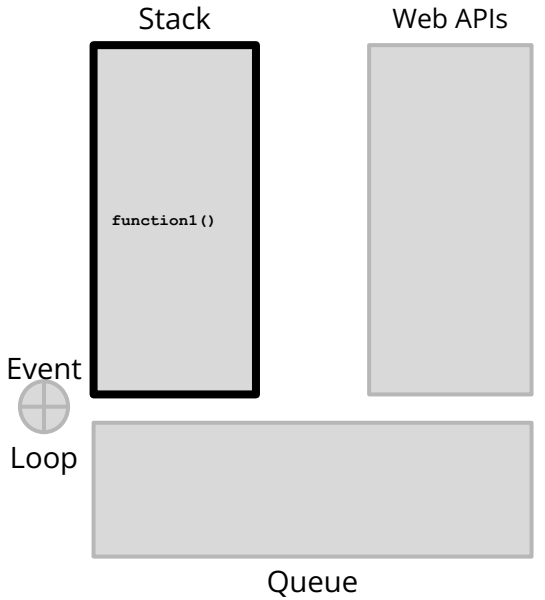
```
function1{
      //code1
}
runFunctionAfterTimeout(function1,2); //timeout = 2’’; assume a really slow machine
function2(){
      //code2
}
function3(){
      //code2
}

function2();
function3();
```



Example

t=-

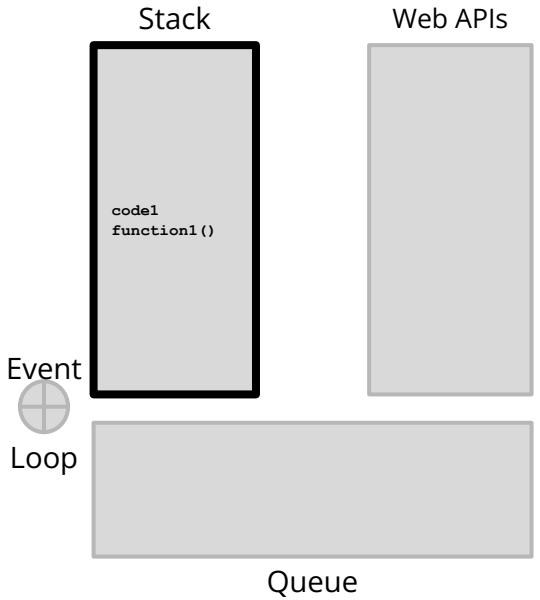
```
function1{
      //code1
}
runFunctionAfterTimeout(function1,2); //timeout = 2’’; assume a really slow machine
function2(){
      //code2
}
function3(){
      //code2
}

function2();
function3();
```



Example

t=-

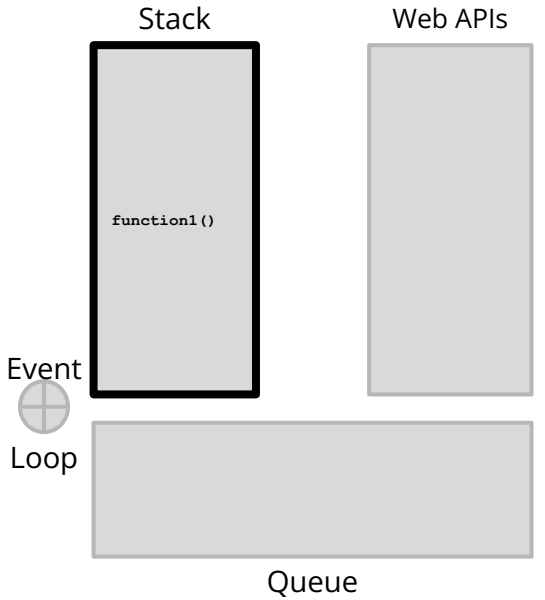
```
function1{
      //code1
}
runFunctionAfterTimeout(function1,2); //timeout = 2''; assume a really slow machine
function2(){
      //code2
}
function3(){
      //code2
}

function2();
function3();
```



Example

t=-

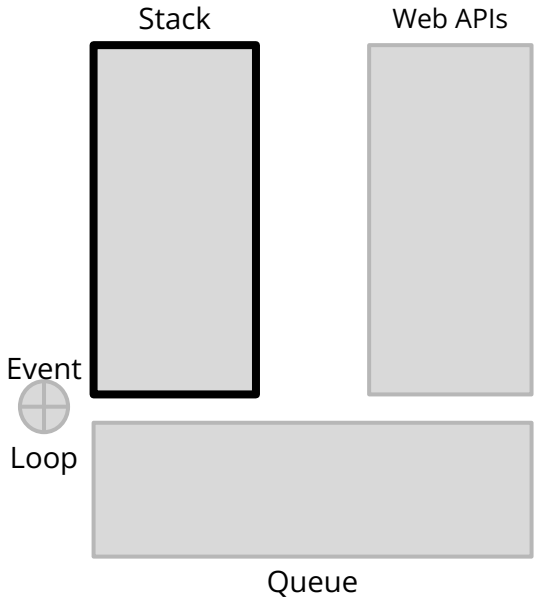
```
function1{
      //code1
}
runFunctionAfterTimeout(function1,2); //timeout = 2’’; assume a really slow machine
function2(){
      //code2
}
function3(){
      //code2
}

function2();
function3();
```



Example

t=-

```
function1{
      //code1
}
runFunctionAfterTimeout(function1,2); //timeout = 2’’; assume a really slow machine
function2(){
      //code2
}
function3(){
      //code2
}

function2();
function3();
```



Characteristics

- curly-bracket syntax
  - Like Java, C
- dynamic typing
  - Unlike Java, like Python
- prototype-based object-orientation
  - Like Java
- first-class functions (treats functions as first-class citizens, e.g. can assign them to variables)
  - Like Java Lambda Expressions, Unlike C
- high-level
  - Unlike Assembly, Like Python
- Interpreted with just-in-time (JIT) compilation
  - Profilers check the code for warm/hot chunks and create and compile stubs

Variables

- Case-sensitive names
- Types are not explicitly declared
- Types are calculated during assignments at run-time
- Primitive types:
  - string, number, bigint, boolean, undefined, and symbol
  - pass by value
- The value of unassigned variables is “undefined”
- Objects can also be assigned to variables
  - pass by reference
- A special object: “null”
  - It’s value is “undefined”
  - Any structured type is derived from “null”

More general (and for what is worth):
JavaScript uses call-by-sharing

```
let a = [1];
let b = a;       // aliasing

b.push(2);       // mutation
b = [3];         // rebinding

console.log(a); // ?
```

Primitive types

- string, number, boolean
  - ...
- bigint
  - indicates that calculations are performed on numbers whose digits of precision are potentially limited only by the available memory of the host system
- undefined
  - primitive value automatically assigned to variables that have just been declared, or to formal arguments for which there are no actual arguments.
  - the undefined global property represents the primitive value undefined
- symbol
  - built-in object whose constructor returns a symbol primitive that's guaranteed to be unique

Scope

- Declarations using the keyword “var”
  - function-scoped variables
- Declarations using the keywords “let” and “const” (ECMAScript 6)
  - block-scoped variables
- Declarations with no keyword
  - globally-scoped variables

Function-scoped variables

```
function f() {
   var x=0;
   console.log(x); //logs "0"
   console.log(y); //logs "6"
}
var y=6;
f();
```

Block-scoped variables

```
   {
      let y=6;
   }
   console.log(y); //logs "ReferenceError: Can't
find variable: y"
```

```
function func() {
      let foo = 5;
      if (···) {
            let foo = 10; // shadows outer `foo`
            console.log(foo); // 10
      }
      console.log(foo); // 5
}
```

Let Vs Const

- Variables created by let are mutable

```
let foo = 'abc';
foo = 'def';
console.log(foo); // def
```

- Variables created by const, constants, are immutable

```
const foo = 'abc';
foo = 'def'; // TypeError
```

Globally-scoped variables

```
function(){
    console.log(x); //logs ‘3’
    x=6;
    console.log(x); //logs ‘6’
}
x=3;
f();
```

Idea

- You can think of variables as a structure with 5 fields
  - Name
  - Address
  - Value (always as text)
  - Type
  - Scope

Quiz

```
var a = "1200";
var b = 1200;
alert(a == b);
//alerts ???
alert(a === b);
//alerts ???
```

Typecasting/Type conversion

- Explicit: The source code requires a conversion
- Implicity: Runtime automatically convert data types
  - Boolean context: coerce any value to a Boolean in contexts that require it
    - E.g., conditionals and loops

```
let a=0;
if(a){
//is this part
executed?
}
```

Implicit typecasting in a boolean context

- **Falsy values**: evaluate to false in a boolean context
  - false, 0, "" (empty string), null, undefined, and NaN.
- **Truthy values**: evaluate to true in a boolean context
  - Everything else, including non-empty strings, numbers other than zero, objects, and arrays.

Falsy values

Value	Type	Description
null	Null	The keyword null - the absence of any value.
undefined	Undefined	undefined - the primitive value.
false	Boolean	The keyword false.
NaN	Number	NaN - not a number.
0	Number	The Number zero, also including 0.0, 0x0, etc.
-0	Number	The Number negative zero, also including -0.0, -0x0, etc.
0n	BigInt	The BigInt zero, also including 0x0n, etc. Note that there is no BigInt negative zero - the negation of 0n is 0n.
""	String	Empty string value, also including '' and ``.
document.all	Object	The only falsy object in JavaScript is the built-in document.all.

Falsy values

Underflow occurs when the result of a numeric operation is closer to zero than the smallest representable number.
In this case, JavaScript returns 0. If underflow occurs from a negative number, JavaScript returns a special value known as “negative zero.” (x → 0−)
[IEEE 754 floating-point arithmetic)

Value	Type	Description
null	Null	The keyword null - the absence of any value.
undefined	Undefined	undefined - the primitive value.
false	Boolean	The keyword false.
NaN	Number	NaN - not a number.
0	Number	The Number zero, also including 0.0, 0x0, etc.
-0	Number	The Number negative zero, also including -0.0, -0x0, etc.
0n	BigInt	The BigInt zero, also including 0x0n, etc. Note that there is no BigInt negative zero - the negation of 0n is 0n.
""	String	Empty string value, also including '' and ``.
document.all	Object	The only falsy object in JavaScript is the built-in document.all.

How to know if foo is declared?

Very common in the event-driven context! E.g., a user presses a button before doing something else

- `if (!foo)` checks the ‘value’ part of the variable foo then it evaluates to false with **falsy** values (ref: https://developer.mozilla.org/en-US/docs/Glossary/Falsy)
- ...so it doesn’t work all of the times
- Idea! Let’s check the ‘type’ part of the variable (`typeof` operator):

```
let foo;
foo==true //false
typeof foo=='undefined' //true
```

declaration	name	address	value	type
let foo;	'foo'	0x1235	'undefined'	'undefined'
	'foo'			'undefined'
let foo=0;	'foo'	0x1235	'0'	'number'
let foo='';	'foo'	0x1235	''	'string'
let foo=NaN;	'foo'	0x1235	'NaN'	'number'
let foo=false;	'foo'	0x1235	'false'	'boolean'
let foo=null;	'foo'	0x1235	'undefined'	'object'

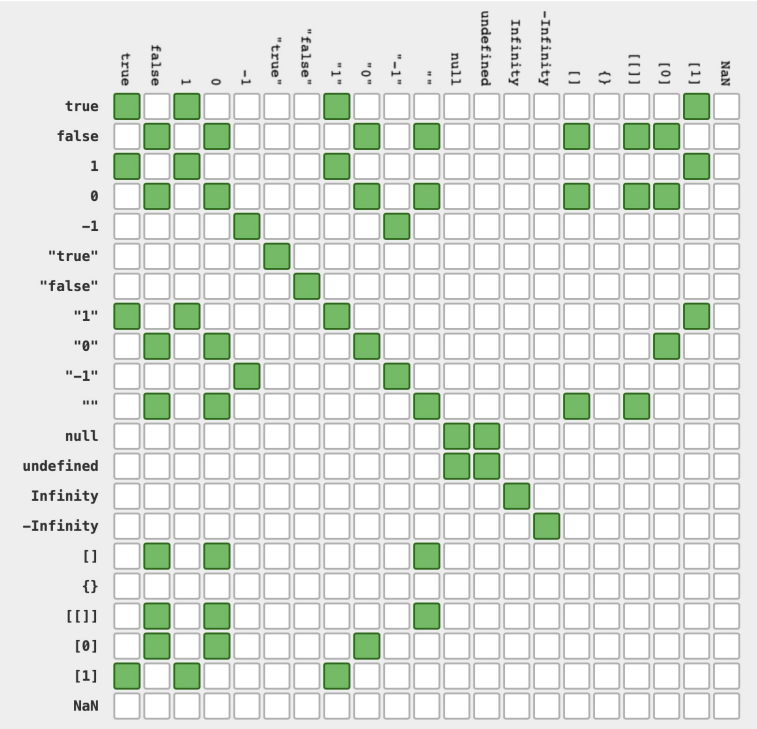
Equality

- == (loose equality)
  - Checks only value (allows coercion)
- === (strict equality)
  - Checks value (no coercion) and type

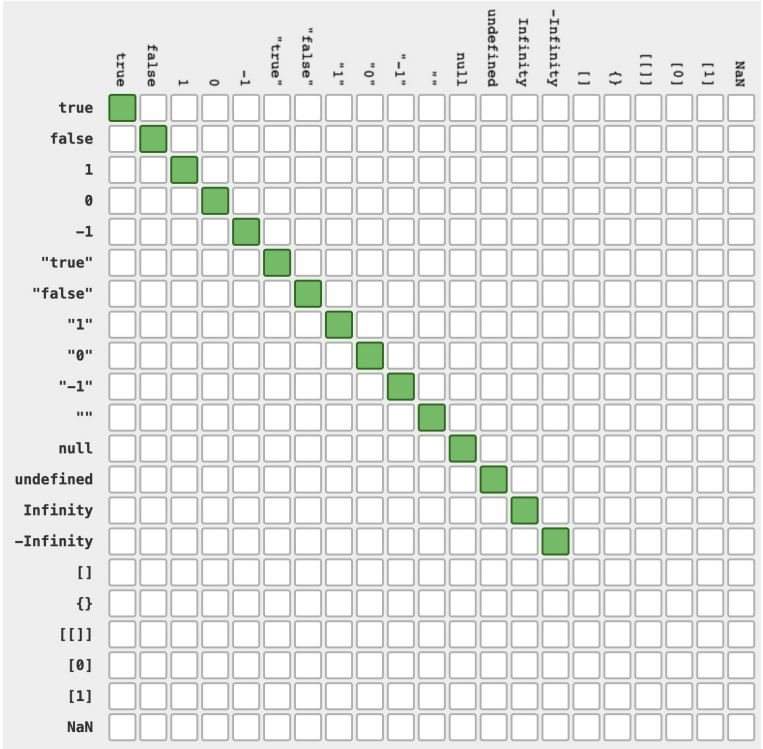
```
0 == false    // true because 0 is coerced to boolean false
0 === false   // false
```

source: https://dorey.github.io/JavaScript-Equality-Table/

Equality Matrix



== (loose equality-allows coercion)



=== (strict equality)

Quiz

- alert(typeof(null));
- alert(typeof(undefined));
- alert(null !== undefined)
- alert(null == undefined)

Quiz

- alert(typeof(null));
  - object
- alert(typeof(undefined));
- alert(null !== undefined)
- alert(null == undefined)

Quiz

- alert(typeof(null));
  - object
- alert(typeof(undefined));
  - undefined
- alert(null !== undefined)

- alert(null == undefined)

Quiz

- alert(typeof(null));
  - object
- alert(typeof(undefined));
  - undefined
- alert(null !== undefined)
  - true
- alert(null == undefined)

Quiz

- alert(typeof(null));
  - object
- alert(typeof(undefined));
  - undefined
- alert(null !== undefined)
  - true
- alert(null == undefined)
  - true

Hoisting

```
var foo = 1;
function bar() {
   if (!foo) {
      var foo = 10;
   }
   alert(foo);
}
bar();
```

alerts?

Hoisting

- Function and variable declarations are always moved (“hoisted”) invisibly to the top of their containing scope by the JavaScript interpreter.

Hoisting

```
var foo = 1;
function bar() {
   if (!foo) {
      var foo = 10;
   }
   alert(foo);
}
bar();
```


```
var foo = 1;
function bar() {
   var foo;
   if (!foo) {
      foo = 10;
   }
   alert(foo);
}
bar();
```

Syntax (control structures)

- IF

```
if(condition1){ … }
else if(condition2){  … }
else{ … }
```

- SWITCH

```
switch(condition){
     case value1:
          ...
          break;
     case value2:
          ...
          break;
     default:
          …
}
```

Syntax (loops)

- For

```
for(initialization;condition;step){
   code
}
```

- While

```
while(condition){
   code
}
```

- Do… While

```
do {
    code
while (condition)
```

Objects

- JavaScript is designed on a simple object-based paradigm
- An object is a collection of properties
  - a property is an association between a name (or key) and a value
  - a property's value can be a function, in which case the property is known as a method.
- Predefined and custom objects
- 4 types to create objects
  - New operator or constructor
  - Object Literals
  - Object.create method
  - Class expression

Instantiating Objects

- Create a new object with the keyword “new”

```
const myCar = new Object();
myCar.make = 'Ford';
myCar.model = 'Mustang';
myCar.year = 1969;
```

Object initialization (Object Literals)

```
const myCar = {
    make: 'Ford',
    model: 'Mustang',
    year: 1969
};

Powerful because:
let m = eval({make: 'Ford',model:'Mustang',year: 1969})
console.log(m.model)    //prints ‘Ford’
```

Objects == Associative Arrays

- Access or set objects using a bracket notation

```
myCar['make'] = 'Ford';
myCar['model'] = 'Mustang';
myCar['year'] = 1969;
```

Execution Context in JS

- The execution context is defined by the stack frame of the currently executing function
- `this` is part of the execution context and it is pointing to the owning object of the currently executing function
- Exceptions:
  - Constructor functions
  - Arrow functions
  - Closures
  - ...

Typical OO (e.g. Java) definition of exec. context and `this`

JS definition of Execution Context is different than most of the languages!
Let's investigate those exceptions

Example

```
function f1() {
  return this;
}
// In a browser:
f1() === window; // true
```

Creating objects using constructor functions

Prototype-based programming

- Define the object type by writing a constructor function.
  - convention: use a capital initial letter
- Create an instance of the object with “new”

```
function Car(make, model, year) {
  this.make = make;
  this.model = model;
  this.year = year;
}
var kenscar = new Car('Nissan', '300ZX', 1992);
var vpgscar = new Car('Mazda', 'Miata', 1990);
```

Methods

```
function displayCar() {
  var result = `A Beautiful ${this.year} ${this.make} ${this.model}`;
  return result;
}
function Car(make, model, year, owner) {
  this.make = make;
  this.model = model;
  this.year = year;
  this.owner = owner;
  this.displayCar = displayCar;
}
var car1 = new Car(...);
var car2 = new Car(...);
console.log(car1.displayCar());
console.log(car2.displayCar());
```

Functions

- Functions are declared with the keyword ‘function’
- They inherit from the Object prototype and they can be assigned key/value pairs
- Functions are first-class citizens in JS (examples)
  - Assign function to a variable
  - Pass function as argument
  - Return a function
    - Using a variable
    - Using double parentheses
- They always return something
  - Whatever the ‘return’ statement defines
  - ‘undefined’ in the absence of the ‘return’ statement
  - Reference to ‘this’ in the case of constructor functions used with the ‘new’ keyword

Expressions for defining functions

- function expression
- Immediately Invoked Function Expression
- function* expression
- Anonymous
- Arrow expression

function expression

```
var myFunction = function(){
    statements
}
```

Immediately Invoked Function Expression (IIFE)

- Useful before ECMAScript6 (function-scoping)
  - avoids variable hoisting from within blocks
  - protects against polluting the global environment because it creates a new scope
  - allows public access to methods while retaining privacy for variables defined within the function

```
(function() {
    statements
})();
```

function* expression

- Defines a ‘generator’ function, which returns a Generator object
- Generator object
  - Conforms to both the iterable protocol and the iterator protocol
  - Instance methods
    - Generator.prototype.next()
      - Returns a value yielded by the yield expression.
    - Generator.prototype.return()
      - Returns the given value and finishes the generator.
    - Generator.prototype.throw()
      - Throws an error to a generator (also finishes the generator, unless caught from within that generator).

function* example

```
function* generator(i) {
  yield i;
  yield i + 10;
}
const gen = generator(10);
console.log(gen.next().value);
// expected output: 10
console.log(gen.next().value);
// expected output: 20
```

Anonymous function expression

```
var myFunction = function() {
    statements
}
```

Object methods using anonymous functions

```
function Car(make, model, year, owner) {
  this.make = make;
  this.model = model;
  this.year = year;
  this.owner = owner;
  this.displayCar = function(){
    var result = `A Beautiful ${this.year} ${this.make} ${this.model}`;
    return result;
  };
}
var car1 = Car(...);
var car2 = Car(...);
console.log(car1.displayCar());
console.log(car2.displayCar());
```

Initialization with anonymous functions as methods

```
let myCar = {
    make:'Nissan',
    model: '300ZX',
    year: 1992,
    owner: 'John',
    displayCar: function(){
        console.log(`A Beautiful ${this.year} ${this.make}
${this.model}`);
    }
};
myCar.displayCar();
```

Arrow function expression

- Compact alternative to traditional function expression
- With limitations and extra properties

```
// Traditional Anonymous Function
function (a){
  return a + 100;
}
// Arrow Function Break Down
(a) => {
  return a + 100;
}
(a) => a + 100;
a => a + 100;
```

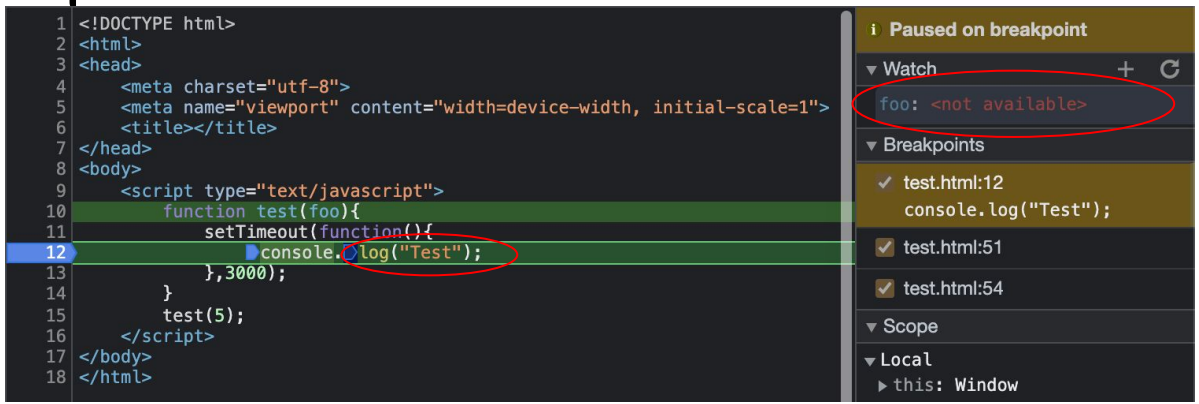
Quiz

```
for(var i = 0; i < 5; i++) {
    setTimeout(function() {
        console.log(i);
    },1000);
}
```

Quiz

```
for(var i = 0; i < 5; i++) {
    setTimeout(function(i) {
        console.log(i);
    },1000);
}
```

Awkward phenomenon



Lexical scoping

```
function init() {
  var name = 'Mozilla'; // name is a local variable created by init
  function displayName() { // displayName() is the inner function, a closure
    alert(name); // use variable declared in the parent function. JS looks up
                 // the scope chain to find a variable called “name”
  }
  displayName();
}
init();  // displays 'Mozilla'
```

Lexical scope describes how nested (also known as "child") functions have access to variables defined in parent scopes. (source)

The lexical scope of displayName function allows access to the parent scope.

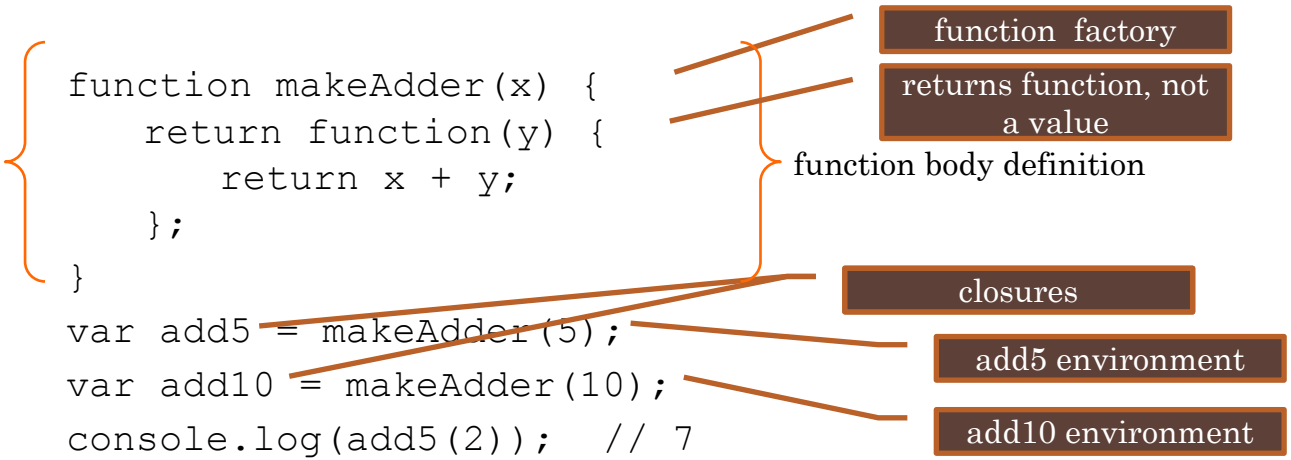
Closures

```
function makeFunc() {
  var name = 'Mozilla';
  function displayName() {
    alert(name);
  }
  return displayName;
}

var myFunc = makeFunc();
myFunc();  // displays 'Mozilla'
```

A closure is a function having access to the parent scope (lexical scoping), even after the parent function has closed. (source)

JavaScript Closures

- A closure is a special kind of object that combines two things:
  - a function
  - the lexical environment in which that function was created
    - The environment consists of any local variables that were in-scope at the time that the closure was created.



https://developer.mozilla.org/en-US/docs/Web/JavaScript/Closures

Closures

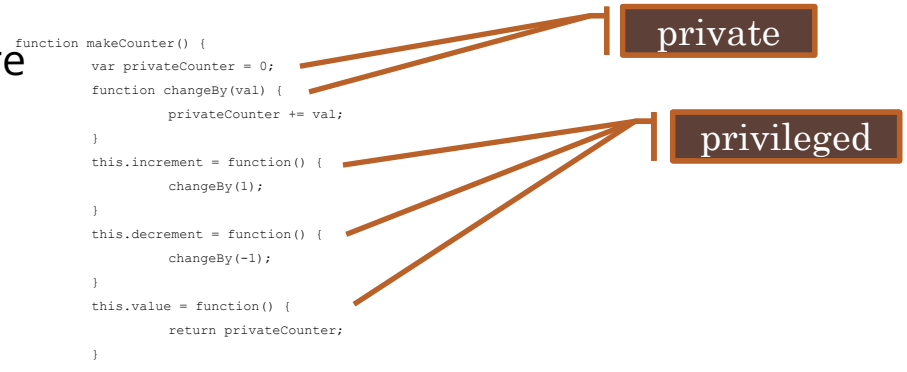
- Definition
  - a closure is a stack frame which is allocated when a function starts its execution, and not freed after the function returns (as if a 'stack frame' was allocated on the heap rather than the stack!).
- Why are they important?
  - They let you associate data (the lexical environment) with a function that operates on that data. Analogous to object-oriented programming, where objects allow you to associate data (properties) with one or more methods.
  - Much of what you do in practice in JavaScript is asynchronous/event driven

Quiz

- Implement a constructor function for an integer Counter
  - Set the starting value and the step. E.g.,
    - `let counter = new Counter(0,1);`
  - Provide methods for increasing or decreasing the counter value. E.g.,
    - `counter.increase()`
  - Provide method for accessing the counter value. E.g.,
    - `console.log(counter.getValue());`

Emulating private accessors with Closures

- No standard way to declare private methods




```
function makeCounter() {
        var privateCounter = 0;
        function changeBy(val) {
                privateCounter += val;
        }
        this.increment = function() {
                changeBy(1);
        }
        this.decrement = function() {
                changeBy(-1);
        }
        this.value = function() {
                return privateCounter;
        }
}
var Counter1 = new makeCounter();
var Counter2 = new makeCounter();
alert(Counter1.value()); /* Alerts 0 */
Counter1.increment();
Counter1.increment();
alert(Counter1.value()); /* Alerts 2 */
Counter1.decrement();
alert(Counter1.value()); /* Alerts 1 */
alert(Counter2.value()); /* Alerts 0 */
alert(Counter1.privateCounter); /* Alerts undefined */
Counter1.changeBy(3); /* Alerts TypeError: Counter1.changeBy is not a function */
```

Singleton pattern and closures

```
const Singleton = (function () {
  let instance; // this variable is closed over

  function createInstance() {
    const object = new Object("I am the instance");
    return object;
  }

  return {
    getInstance: function () {
      if (!instance) {
        instance = createInstance(); // instance is created only once
      }
      return instance;
    },
  };
})();

const instance1 = Singleton.getInstance();
const instance2 = Singleton.getInstance();
console.log(instance1 === instance2); // true
```

Reminder

- A **singleton** pattern ensures that only one instance of a class or object is created and provides a global access point to it.
- A **closure** happens when a function "remembers" the variables from its outer (enclosing) scope, even after that outer function has finished executing.

JS “formal” definitions of `this` and execution context

`this` keyword (non-strict mode)

- The value of **`this`** is the object that is used to call the function
  - It is NOT the object that has the function as an own property
- The value of **`this`** is determined by how a function is called (runtime binding)
- Cases:
  - When a function is called as a method of an object (e.g., object.method()), **`this`** refers to the object.
  - When a function is called as a standalone function (e.g., functionName()), **`this`** typically refers to the global object (window in a browser environment).
  - When a function is used as a constructor with the new keyword, **`this`** refers to the newly created object.
  - When a function is called with methods like call(), apply(), or bind(), you can explicitly set the value of **`this`**.

Execution context

- The term "execution context" refers to the environment in which the code is executed.
- The execution context is created in two main situations:
  - **Global Execution Context**: This is the outermost execution context and is created when your code is first loaded or run. It includes all global variables and functions. In a web browser, the global object is typically **`window`**.
  - **Function Execution Context**: A new execution context is created each time a function is called. This context includes the local variables and parameters of the function. It also has a reference to the outer environment, allowing access to variables from the enclosing scope.
- It consists of two main components:
  - Variable Environment
  - Lexical Environment

Execution context: Variable Environment

- The Variable Environment contains variables and functions declared within the current context (e.g., function or block).
- It includes the function's formal parameters, local variables, and any function declarations (function declarations are hoisted).
- It also contains a reference to the outer (enclosing) environment, allowing access to variables from the outer scope.
- The **`this`** value, which can change depending on how a function is called, is part of the Variable Environment.

Execution context: Lexical Environment

- The Lexical Environment is a slightly broader concept that encompasses the Variable Environment.
- It includes the Variable Environment, but it also stores information about the lexical structure of the code, such as the scope chain.
- The scope chain is a list of Variable Environments, representing all the nested contexts (lexical scopes) that can be accessed to resolve variables.

Execution Context: Example

```
var globalVar = 10;
function outer() {
  var outerVar = 20;
  function inner() {
    var innerVar = 30;
    console.log(globalVar + outerVar + innerVar);
  }
  inner();
}
outer();
```

‘this’ and Arrow functions: Traditional functions

```
window.age = 10;
function Person() {
  this.age = 42;
  setTimeout(function () {
    console.log("this.age", this.age); // yields "10" because the
function executes on the window object scope
  }, 100);
}

var p = new Person();
```

‘this’ and Arrow functions Vs Traditional functions

```
window.age = 10;
function Person() {
  this.age = 42;
  setTimeout(() => {
    console.log("this.age", this.age); // yields "42" because the
function executes on the Person scope
  }, 100);
}

var p = new Person();
```

Inheritance and ES06 `class` expression

Inheritance

- All JS objects inherit from Object
- All objects include a ‘prototype’ property
    - a bucket for storing properties and methods when we are extending a class

```
function Person(first, last, age, gender, interests) {
  this.name = {
    first,
    last
  };
  this.age = age;
  this.gender = gender;
  this.interests = interests;
};
Person.prototype.greeting = function() {
  alert('Hi! I\'m ' + this.name.first + '.');
};
```

Inheriting objects with ‘call’

```
function Person(first, last, age, gender, interests) {
  this.name = {
    first,
    last
  };
  this.age = age;
  this.gender = gender;
  this.interests = interests;
  this.greeting = function() {
    alert('Hi! I\'m ' + this.name.first + '.');
  };
};
function Teacher(first, last, age, gender, interests, subject) {
  Person.call(this, first, last, age, gender, interests);
  this.subject = subject;
}
let teacher = new Teacher('John','Doe',30,'male','history','art history');
teacher.greeting();
```

ES06 Class expression

- Syntax

```
const MyClass = class [className] [extends otherClassName] {
  // class body
}
```

- Example

```
const Foo = class {
  constructor() {}
  bar() {
    return 'Hello World!';
  }
};
const instance = new Foo();
instance.bar();  // "Hello World!"
Foo.name;        // "Foo"
```

Quiz

- Do the previous quiz but this time use the “class” expression

Class expression: extends & private methods

```
class ClassWithPrivateField {
  #privateField;

  constructor() {
    this.#privateField = 42;
  }
}

class SubClass extends ClassWithPrivateField {
  #subPrivateField;

  constructor() {
    super();
    this.#subPrivateField = 23;
  }
}

new SubClass();
// SubClass {#subPrivateField: 23}
```

Iterating through object properties

- for-in loop (valid for Object Literals)

```
for (const variable_name in object_name){
//statement (object_name[variable_name])
}
```

- Example

```
for (const prop_name in teacher)
    console.log(teacher[prop_name]);
```

Quiz

```
var a = 5;
(function() {
  var a = 12;
  alert(a);
})();
```

```
var a = 10;
var x = (function() {
  var y = function() {
    var a = 12;
  };
  return function() {
    alert(a);
  }
})();
x();
```

```
var a = 10;
(function() {
  var a = 15;
  window.x = function() {
    alert(a);
  }
})();
x();
```

JS for client-side programming

JS in client-side programming

- Integrated in HTML

```
<html>
    <body>
    <script type="text/javascript">
        document.write("Hello World!");
    </script>
    </body>
</html>
```

Executing JS in HTML

- Time of execution depends on their position in the page
- Strong event-driven programming element
- Declaring functions in <head> is a good practice
- You can always refer to an external JS file

```
<script src="code.js"></script>
```

Quiz

- Open a text editor and save the file as “test.html”
- Write a script within the HTML code, that will print the numbers 1-99, omitting one of them
  - In the browser’s console
  - In the web page that it will load
- Run the test.html in your browser

Events

- User-activity-related events
  - Mouse/keyboard actions
- Time-related events
  - `setTimeout(function, timeout)`
  - `setInterval(function, interval)`
- System-related events
  - Page loads, form submitted, I/O-related events

Example events

Attribute	The event occurs when...
onblur	An element loses focus
onchange	The content of a field changes
onclick	Mouse clicks an object
ondblclick	Mouse double-clicks an object
onerror	An error occurs when loading a document or an image
onfocus	An element gets focus
onkeydown	A keyboard key is pressed
onkeypress	A keyboard key is pressed or held down
onkeyup	A keyboard key is released
onmousedown	A mouse button is pressed
onmousemove	The mouse is moved
onmouseout	The mouse is moved off an element
onmouseover	The mouse is moved over an element
onmouseup	A mouse button is released
onresize	A window or frame is resized
onselect	Text is selected
onunload	The user exits the page
Property	**Description**
altKey	Returns whether or not the "ALT" key was pressed when an event was triggered
button	Returns which mouse button was clicked when an event was triggered
clientX	Returns the horizontal coordinate of the mouse pointer when an event was triggered
clientY	Returns the vertical coordinate of the mouse pointer when an event was triggered
ctrlKey	Returns whether or not the "CTRL" key was pressed when an event was triggered
metaKey	Returns whether or not the "meta" key was pressed when an event was triggered
relatedTarget	Returns the element related to the element that triggered the event
screenX	Returns the horizontal coordinate of the mouse pointer when an event was triggered
screenY	Returns the vertical coordinate of the mouse pointer when an event was triggered
shiftKey	Returns whether or not the "SHIFT" key was pressed when an event was triggered
Property	**Description**
bubbles	Returns a Boolean value that indicates whether or not an event is a bubbling event
cancelable	Returns a Boolean value that indicates whether or not an event can have its default action prevented
currentTarget	Returns the element whose event listeners triggered the event
eventPhase	Returns which phase of the event flow is currently being evaluated
target	Returns the element that triggered the event
timeStamp	Returns the time stamp, in milliseconds, from the epoch (system start or event trigger)
type	Returns the name of the event

Managing Events in the client

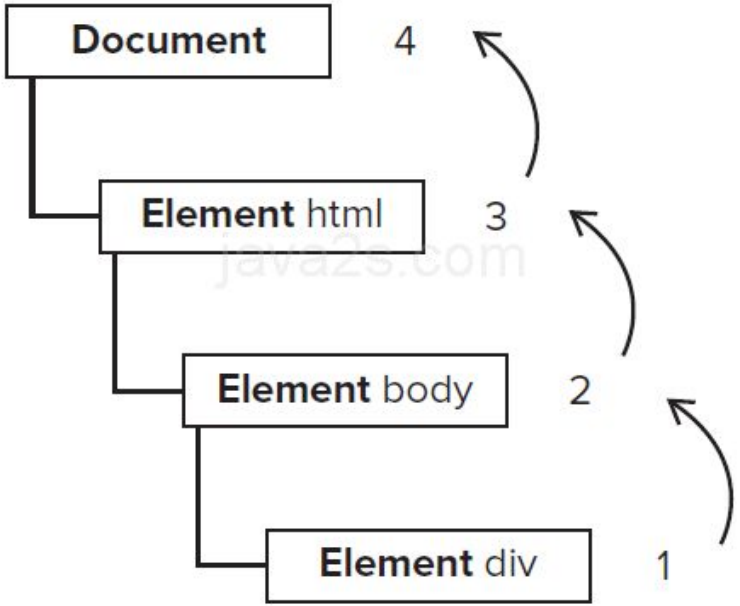
- Create *onevent handlers* in the HTML elements
  - `<input type=”button” value=”OK” onclick=”return function1();”/>`
  - Event handlers are HTML attributes
  - Mixing HTML and JS and without `<script>`
- Use `addEventListener` and a callback
  - target.addEventListener(type, listener [, options]);

Quiz

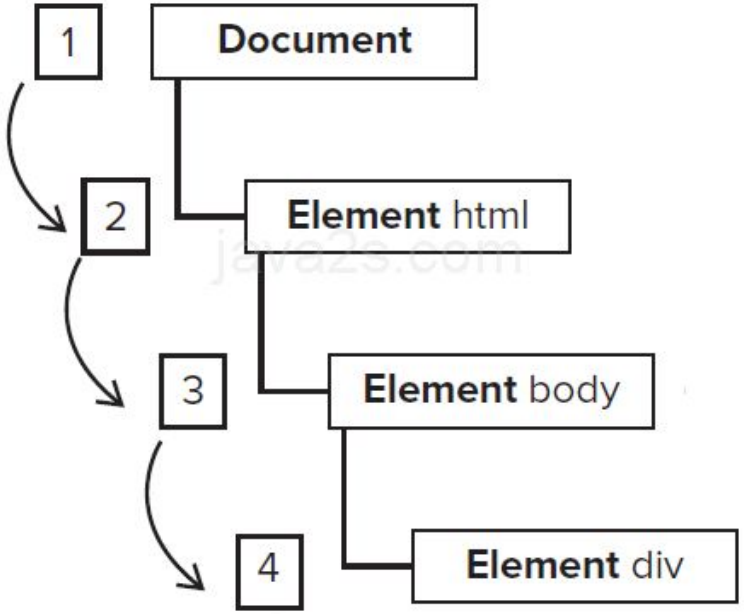
- Any question about the ordering of fired events?

Event order

Bubbling



Capturing



Quiz

- Create a webpage with a textbox (input type=”text”) that when clicked it will show a popup (alert()) stating something
- Create a link (<a href=”http://...”) that will do the same when the mouse cursor hovers over it (event: mouseover)

JS Reference Objects

- JS Reference
  - Array
  - Boolean
  - Date
  - Error
  - JSON
  - Math
  - Number
  - Operators
  - RegExp
  - Statements
  - String
- HTML Element Objects Reference
  - ...

- **Web APIs**
  - Console
  - Geolocation
  - History
  - Storage
  - DOM
  - ...

Quiz

- Write a script that will create an object with three properties (numbers) and a method that once invoked it will print in the console the sum of the numbers
  - Using initialization (Object Literals)
  - Using a constructor function
  - Using the class expression

Sources

- https://developer.mozilla.org/en-US/docs/Web/
- https://www.w3schools.com/